# LAB ASSIGNMENT 5
# Taxi Route Optimisation using Tabular Q-Learning

Akhil Sebastian(BL.SC.P2DSC25002)

1. **Objective:** To explore an application-oriented Reinforcement Learning (RL) problem, formulate it using RL components (agent, environment, state, action, reward), implement QLearning to train an agent, and analyse the performance of the trained agent through learning curves and quantitative measures.

## 2. Application Description:

Application / domain: Taxi route optimization is a classic navigation and logistics problem, modelled here by the Gymnasium Taxi environment (a 5×5 grid world). It is a simplified version of real-world ride-hailing and delivery dispatch.

Problem to be solved: A taxi starts at a random cell. A passenger waits at one of four fixed locations (R, G, Y, B) and has a destination among those same four locations. The taxi must drive to the passenger, pick them up, drive to the destination, and drop them off, while avoiding walls and illegal pickup/drop-off actions.

Objective of the RL agent: Learn a policy that completes the pickup-and-drop-off task in as few steps as possible, i.e. maximise cumulative reward (reach the destination quickly with no illegal actions).

## 3. RL Problem Formulation:

| Component | Description |
|---|---|
| Agent | The taxi driver (learning controller) that selects a movement or pickup/drop-off action at each time step. |
| Environment | Gymnasium Taxi-v4 (same dynamics as Taxi-v3): a 5×5 grid with internal walls, four designated stops R, G, Y, B, one passenger and one destination. |
| State space | Discrete, 500 states = 25 taxi positions × 5 passenger locations (R, G, Y, B, or in the taxi) × 4 destinations. Encoded as an integer 0–499. |
| Action space | Discrete, 6 actions: 0 = South, 1 = North, 2 = East, 3 = West, 4 = Pickup, 5 = Drop-off. |
| Reward function | −1 for every time step (encourages short routes); +20 for a successful drop-off at the correct destination; −10 for an illegal Pickup or Drop-off action. |

| | |
|---|---|
| Terminal condition | Episode terminates on a successful drop-off. Episode is truncated after 200 steps (time limit). |

## 4. Algorithm Used: Q-Learning:

### 4.1 Working principle

Q-Learning is a model-free, off-policy, value-based temporal-difference (TD) algorithm. It learns an action-value function Q(s, a), the expected discounted return of taking action a in state s and acting optimally afterwards. With 500 states and 6 actions, Q is stored as a 500×6 table that is updated after every step:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \right]$$

Actions during training are chosen with an ε-greedy policy: with probability ε a random action is taken (exploration), otherwise the action with the highest Q-value is taken (exploitation). ε is decayed over time so that the agent explores widely at first and exploits what it has learned later. It is off-policy because the update uses the max over next actions, not the action actually taken.

### 4.2 Why Q-Learning is suitable

- The state and action spaces are small and discrete (500 × 6), so an exact Q-table fits in memory and no function approximation (DQN) is needed.
- The environment dynamics are unknown to the agent and are learned purely from experience (model-free).
- Q-Learning converges to the optimal action-value function when every state-action pair is visited sufficiently often and the learning rate is suitable.
- It learns the optimal (greedy) policy regardless of the exploratory behaviour policy, unlike SARSA, which learns the value of the exploring policy. **4.3 Training parameters**

| Parameter | Value | Role |
|---|---|---|
| **Learning rate (α)** | 0.1 | Step size of the Q-value update. |
| **Discount factor (γ)** | 0.99 | Weights future rewards; high value because the task needs a multi-step plan. |
| **Initial ε** | 1.0 | Fully random at the start for maximum exploration. |
| **Minimum ε** | 0.05 | Keeps a small amount of exploration throughout. |
| **ε decay** | 0.999 / episode | ε reaches its minimum after about 3,000 episodes. |
| **Training episodes** | 5000 | Total episodes used for learning. |
| **Max steps per episode** | 200 | Environment time limit. |

| | | |
|---|---|---|
| **Q-table initialisation** | All zeros | Ties broken randomly between equal Q-values. |
| **Random seed** | 42 | Reproducibility. |

## 5. Implementation:

The agent is implemented in Python using NumPy for the Q-table, Gymnasium for the Taxi environment and Matplotlib for plots. Main components:

- epsilon_greedy(state, eps): selects a random action with probability ε, otherwise a greedy action with random tie-breaking.
- Training loop: for each episode, reset the environment, step until termination or truncation, apply the Q-Learning update after every step, record episode reward, steps and success, then decay ε.
- Evaluation loop: after training, run 1,000 episodes with the purely greedy policy (ε = 0) and no learning.
- Outputs: results.json (metrics), learning_curve.png, reward_vs_episodes.png, q_table.npy and demo_episode.txt (text rendering of one trained episode).

## 6. Training Details:

| Item | Value |
|---|---|
| **Number of training episodes** | 5000 |
| **Average training reward (all episodes)** | -70.83 |
| **Best / worst single training episode** | 15 / -920 |
| **Average reward, first 100 episodes** | -752.42 |
| **Average reward, last 100 episodes** | 6.09 |
| **Average steps/episode, first 100 → last 100** | 198.9 → 13.4 |
| **Success rate, first 100 → last 100 episodes** | 4% → 100% |

```
$ python taxi_qlearning.py

Episodes trained           : 5000
alpha / gamma              : 0.1 / 0.99
epsilon                    : 1.0 -> 0.05 (decay 0.999)

First 100 eps  avg reward  : -752.42   success: 4%   steps: 198.9
Last 100 eps   avg reward  : 6.09    success: 100%  steps: 13.4
Greedy eval (1000 eps)    : avg reward 8.12 +/- 2.60
                             success 100%   avg steps 12.88

Avg reward per 500-episode block:
  -559.9, -146.2, -26.4, -4.7, 2.4
  4.7, 5.5, 5.3, 5.4, 5.7
```

Figure 1: Console output of the training run (screenshot).

## **7.** Results and Learning Curve

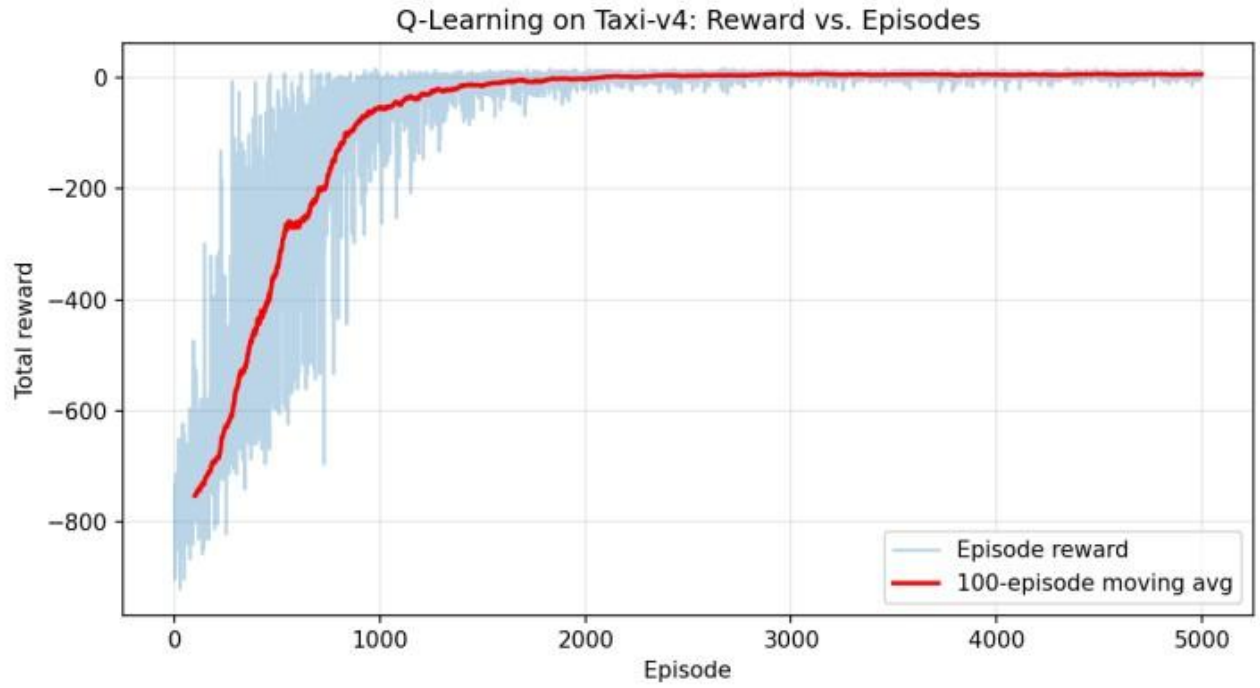



Figure 2: Reward vs. Episodes (raw reward and 100-episode moving average).

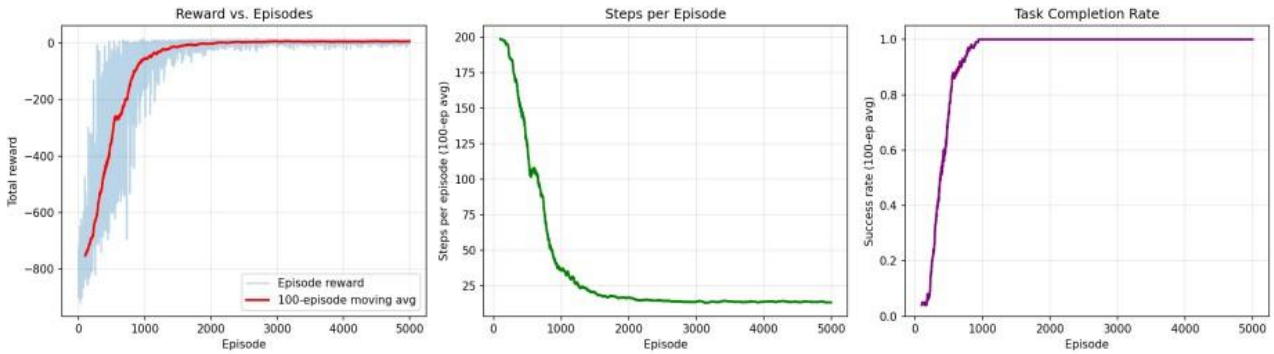



Figure 3: Reward, steps per episode and task-completion rate during training.

### 7.1 Final performance (greedy policy, no exploration):

| Metric | Result over 1000 evaluation episodes |
|---|---|
| **Average reward** | 8.12 ± 2.60 (std) |
| **Success / task-completion rate** | 100% |
| **Average steps to complete task** | 12.88 |

### 7.2 Demonstration of a trained episode:

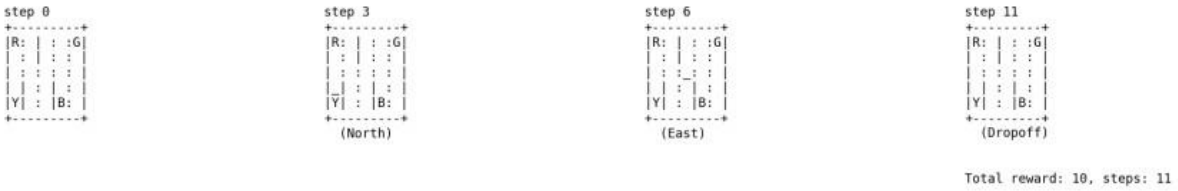



Figure 4: Selected frames of one trained episode (taxi path to the passenger, then the drop-off).

## 8. Performance Analysis:

### 8.1 Average reward by training stage

| Episodes | Average reward |
|---|---|
| **1 – 500** | -559.93 |
| **501 – 1000** | -146.20 |
| **1001 – 1500** | -26.41 |
| **1501 – 2000** | -4.72 |
| **2001 – 2500** | 2.40 |
| **2501 – 3000** | 4.71 |
| **3001 – 3500** | 5.46 |
| **3501 – 4000** | 5.32 |
| **4001 – 4500** | 5.35 |
| **4501 – 5000** | 5.68 |

Does performance improve with training?

Yes. The average reward rises from -752.42 in the first 100 episodes to 6.09 in the last 100. The agent's average episode length falls from 198.9 steps (essentially always hitting the 200-step limit) to 13.4 steps, and the task-completion rate rises from 4% to 100%.

Training behaviour.
Learning shows three phases.

(1) Episodes 1–~1,000: ε is high, behaviour is almost random, and rewards are strongly negative because of the −1 per step and frequent −10 illegal pickups/drop-offs.
(2) Episodes ~1,000–~2,500: the Q-values propagate backwards from the +20 drop-off reward, the moving average rises steeply and episodes become shorter.
(3) After ~2,500–3,000 episodes: the curve plateaus around +5 to +6 as the policy converges.

Training vs. evaluation reward:
The last-100 training reward (6.09) is lower than the greedy evaluation reward (8.12) because training still uses ε = 0.05, so about 5% of actions are random and occasionally cause detours or penalties. The greedy policy used in evaluation has no such noise.

Quality of the learned policy:
The greedy agent completes 100% of the 1,000 evaluation episodes in about 12.9 steps on average, with an average reward of about 8.1. This is close to the best achievable for this environment (about 8), which indicates the Q-table has converged to a near-optimal policy.

Variance:
The remaining standard deviation in evaluation reward reflects the different random start positions, passenger locations and destinations, which produce different shortest-path lengths, not unstable behaviour.

**9. Observations**

- Early training is dominated by random exploration and illegal-action penalties; the first successful drop-offs are rare (4% of the first 100 episodes).
- Once a few successful drop-offs occur, the +20 reward propagates backward through the Q-table, and improvement becomes rapid.
- The 100-episode moving average is smooth and monotonic in trend, while the raw episode reward is noisy because of ε-greedy exploration and random initial conditions.
- The ε floor of 0.05 prevents the training reward from reaching the optimum; greedy evaluation is the right measure of final policy quality.
- Results depend on hyper-parameters. A smaller ε decay (faster) would converge sooner but risks under-exploration; a much smaller α would learn more slowly. Only one seed was run here; repeating over several seeds would give confidence intervals.

**10. Conclusion**

Q-Learning with an ε-greedy policy successfully solved the Taxi route optimization problem. After 5000 training episodes, the agent achieves a 100% task-completion rate with an average greedy reward of 8.12 in about 12.9 steps. The learning curve confirms clear improvement in reward, episode length and success rate as training progresses. The tabular

approach is well suited to this small discrete problem; larger or continuous problems such as autonomous driving or CartPole would require function approximation (DQN).